

AI Drawing Review Comment Analyzer & Intelligence Hub

Live Demonstration Guide & Hands-On Dataset Presentation Script

1. Demonstration Overview & Team Responsibilities

This guide structures the live evaluation session for company leadership. The demonstration follows a real sample drawing from the UCC dataset (dataset/raw_drawings) through the entire four-stage pipeline. Each team member operates the live application, shows their specific screen, and demonstrates their direct contribution.

Order & Member	Demonstration Scope & Screens	Hands-On Dataset Action	Individual Responsibility
1. Varad Frontend UI Lead	- Dashboard Screen - Upload Drawing Screen	- Review real-time KPI cards - Load sample drawing from dataset - Use Recent Files & Dept auto-selection - Trigger background processing	PySide6 GUI architecture, theme engine, responsive layout, drag-and-drop ingestion, and executive dashboard.
2. Swapnaja AI / ML Lead	- Comment Viewer Canvas - OCR Results Screen	- Zoom/pan on redline revision clouds - Interactive box selection & inspector focus - Handwritten OCR extraction review - Live 'Clean Text' symbol normalization	Computer vision redline segmentation, title block exclusion, hybrid OCR, text cleaning service, and DistilBERT NLP classification.
3. Soham System Architect	- Human Review Screen - Settings (Category Editor)	- Demonstrate 'Needs Attention' triage - Show [Auto-Approved (AI)] tags - Execute live human override (Reject) - Add new error category in Settings	MVC system architecture, AppController orchestration, WorkflowEngine multithreading, and autonomous triage logic (>=85%).
4. Satyajeeet Database Lead	- Audit History Panel - Analytics Screen - Export Report Screen	- Expand immutable audit log with override - Demonstrate per-department Pareto charts - Populate UCC project metadata - Generate & open live Excel Error Tracker	SQLite relational schema, multi-department taxonomy, immutable audit logging, WAL mode concurrency, and Excel export engine.

Part 1: Varad -- Frontend UI Architecture & Ingestion

Estimated Time: ~4 mins | **Contribution:** PySide6 GUI architecture, theme engine, responsive layout, drag-and-drop ingestion, and executive dashboard.

[Live Screen Actions with Dataset]:

- Launch application in full-screen dark theme; display live Executive Dashboard.
- Hover over KPI summary cards (Total Comments, Approved, Rejected, Flagged).
- Navigate to 'Upload Drawing' screen via left navigation sidebar.
- Open 'Recent Drawings' dropdown and select a pre-processed blueprint (e.g., Piping Drawing).
- Highlight the automatic pre-selection of the Engineering Department ('Piping Engineering').
- Click 'Start Processing' and draw attention to the responsive, multithreaded progress bar.

[Spoken Script -- Varad]:

"Good morning, everyone. I am Varad, and I was responsible for the **Frontend UI Architecture and User Experience** of our system. My goal was creating a responsive, high-DPI desktop interface tailored specifically for UCC engineers that requires zero training.

Right now, you are looking at our **Executive Dashboard**. At the top, these 4 dynamic KPI cards provide project leaders with an instant quality snapshot--Total Comments, Approved, Rejected, and Flagged counts--queried directly from our database. On the right, our live Activity Feed tracks recent drawing reviews.

Let us process a real UCC drawing together. Navigating to the **Upload Screen**: engineers can drag and drop any multi-page blueprint. Notice our **Recent Drawings dropdown**: selecting a previously analyzed drawing instantly loads its details and auto-assigns its official engineering department--in this case, **Piping Engineering**.

When I click '**Start Processing**', the heavy computation runs in the background while the UI displays fluid progress updates, completely free of freezing or lag.

Now that the drawing is ingested, let us see how our AI visually locates redline revision clouds and extracts handwriting. I will pass the controls to **Swapnaja**."

Part 2: Swapnaja -- AI / ML, Computer Vision & Classification

Estimated Time: ~5 mins | Contribution: Computer vision redline segmentation, title block exclusion, hybrid OCR, text cleaning service, and DistilBERT NLP classification.

[Live Screen Actions with Dataset]:

- Navigate to 'Comment Viewer' screen on sidebar; display full engineering sheet.
- Use [+] and [-] zoom buttons to zoom into a redline revision cloud on the canvas.
- Click directly on a detected bounding box--demonstrate glowing cyan highlight and inspector auto-focus.
- Navigate to 'OCR Results' screen on sidebar; show raw text extracted from handwriting.
- Locate a comment with engineering notations and click 'Clean Text' to demonstrate symbol normalization.

[Spoken Script -- Swapnaja]:

"Thank you, Varad. Hello everyone, I am Swapnaja, and my responsibility was developing the **AI, Computer Vision, and Natural Language Processing pipeline**. Engineering drawings are full of complex vector lines, text, and title blocks. My task was teaching the system to 'see' and 'read' only the reviewer's markup.

Looking at the **Comment Viewer**: on this large canvas, bounding boxes are automatically drawn around reviewer annotations. Behind the scenes, my computer vision algorithms segment the red ink in HSV color space and analyze the contour geometry of revision clouds and callout boxes, ignoring straight grid lines.

Crucially, we built an envelope filter that **strictly excludes title blocks and approval stamps** in the margins, so non-comment metadata never pollutes the error tracker. Clicking any box auto-centers it and displays its extracted text and confidence score.

Next, on the **OCR Results Screen**: we use a hybrid OCR model capable of interpreting engineering handwriting. Because handwriting can smudge, I developed a specialized **Text Cleaning Service**. When I click '**Clean Text**', it standardizes industrial symbols--correcting diameter marks ($\varnothing$), tolerances ($\pm$), and foot-and-inch dimensions.

Finally, our fine-tuned **DistilBERT language model** classifies each comment into standard UCC categories like Dimensional, Standards, or Drafting.

Now, how does the system turn these extractions into a fast, automated review workflow? I will pass the controls to **Soham**, who engineered our backend architecture and autonomous triage."

Part 3: Soham -- Backend Architecture, Orchestration & Triage

Estimated Time: ~5 mins | **Contribution:** MVC system architecture, AppController orchestration, WorkflowEngine multithreading, and autonomous triage logic (>=85%).

[Live Screen Actions with Dataset]:

- Navigate to 'Human Review' screen on sidebar.
- Highlight the 'View:' dropdown at top right, showing the default 'Needs Attention' mode.
- Switch dropdown to 'All Comments' to display comments bearing the blue [Auto-Approved (AI)] badge.
- Select an auto-approved comment. Point out the 4 operational buttons: Approve, Reject, Flag, Edit.
- Click 'Reject' (R): demonstrate live human override--badge disappears, status updates to Rejected.
- Navigate to 'Settings' -> 'Error Classifications' (tab). Type 'Clearance Hazard' and click 'Add Category'.

[Spoken Script -- Soham]:

"Thank you, Swapnaja. Hello everyone, I am Soham, and I was responsible for the **System Architecture, Controller Orchestration, and the Automated Review Workflow**.

A critical question for UCC was: If engineers still have to click 'Approve' on 60 comments per drawing, does it really feel like automation?

To solve this, I engineered our **Autonomous Triage Pipeline**. Looking at our **Human Review Screen**, the view filter defaults to '**Needs Attention**'. In our backend workflow_engine.py, any comment with **confidence >= 85% is automatically marked as Approved!** The engineer only spends their time inspecting the 5 or 10 tricky exceptions that genuinely require human verification.

If I switch the filter to '**All Comments**', notice this blue badge: **[Auto-Approved (AI)]**. However, the AI never locks you out. All 4 actions--**Approve, Reject, Flag, and Edit**--remain fully active.

Watch this: if I disagree with an auto-approved comment and click '**Reject**', the system immediately overrides the AI, updates the database, and logs a permanent quality record.

UCC also required custom classifications per project. In **Settings** under **Error Classifications**, I can add a new category like '**Clearance Hazard**' live, and it instantly becomes available across all menus without restarting.

Now, how do we track this audit trail, analyze department trends, and generate UCC's official Excel sheet? I will hand over to **Satyajeet**."

Part 4: Satyajeet -- Database Architecture, Analytics & Export

Estimated Time: ~5 mins | **Contribution:** SQLite relational schema, multi-department taxonomy, immutable audit logging, WAL mode concurrency, and Excel export engine.

[Live Screen Actions with Dataset]:

- In Human Review screen, click 'Audit History' to expand the immutable audit ledger.
- Point to Soham's recent override action recorded with timestamp, reviewer ID, old status, and new status.
- Navigate to 'Analytics' screen; switch Department dropdown from 'Piping' to 'Structural & Physical Design'.
- Demonstrate the live recalculation of the Pareto Chart, monthly trend curve, and donut chart.
- Navigate to 'Export Report' screen; fill Contract # (CTR-2026-01), Plant Name, and WO #.
- Click 'Export Error Tracker Sheet' and open the generated .xlsx spreadsheet on screen.

[Spoken Script -- Satyajeet]:

"Thank you, Soham. Hello everyone, I am Satyajeet, and I was responsible for the **Database Architecture, Multi-Department Data Infrastructure, and the Export Engine.**

In an industrial organization like UCC, drawing observations represent valuable corporate intellectual property.

1. Tamper-Proof Audit Trail: Notice this **Audit History panel** right below the comment Soham just rejected. It displays the exact chronological record: who changed it, the exact timestamp, the previous status ('Approved'), and the new status ('Rejected'). This gives UCC complete traceability for client quality audits and ISO compliance.

2. Departmental Pareto Analytics: On the **Analytics Screen**, our database organizes observations across all **7 UCC engineering disciplines** (Piping, Structural, Electrical, GPD, Plakon, Pipe Support, and Systems). Watch as I switch departments: the database instantly recalculates our live **Pareto Chart**. This identifies the vital 20% of error categories causing 80% of rework in each department.

3. One-Click UCC Error Tracker Export: On the **Export Screen**, we enter UCC's standard project metadata--Contract Number, Plant Name, and Work Order. When I click '**Export Error Tracker Sheet**', our engine builds UCC's official formatted spreadsheet.

Opening the file: notice the exact standard formatting--**Orange columns** for user inputs, **Green columns** for auto-extracted fields, and dedicated tabs automatically created for each active department.

4. 100% On-Premise Data Privacy: Everything you have seen today runs **100% locally on your office workstations**. Not a single blueprint or contract ever leaves UCC's internal network.

With that, I will hand back to Soham to conclude."

5. Executive Conclusion & Q&A Handoff

Speaker: Soham (~1 min)

"To conclude our demonstration:

- **Varad** demonstrated how engineers upload, inspect, and navigate drawings effortlessly.
- **Swapnaja** demonstrated how our AI computer vision accurately isolates redline clouds, cleans technical symbols, and classifies engineering errors.
- I demonstrated how our backend autonomous triage cuts manual review time by over 80% while keeping engineers in 100% control.
- And **Satyajeet** demonstrated how all observations are organized into departmental Pareto analytics and exported directly into UCC's official Error Tracker format.

Our system is fully integrated, backed by **92 passing automated unit tests**, and ready for deployment.

Thank you, Vishal Sir and the UCC leadership team. We are now delighted to answer any questions!"

6. Executive Q&A Preparation Cheat-Sheet

Potential Executive Question	Best Respondent	Strategic Response Summary
What if handwriting is messy and OCR misreads it?	Swapnaja (AI Lead)	The model assigns a confidence score. If confidence is below 85%, it is never auto-approved. It is held in the 'Needs Attention' queue for quick 1-click human correction.
Can different departments add their own custom error categories?	Soham / Satyajeet	Yes. The Category Editor in Settings allows adding dynamic categories to the database on the fly without changing a single line of code.
Does the Excel export match our company template?	Satyajeet (Database Lead)	Yes. The export engine uses openpyxl with exact column positioning, color codes (orange for inputs, green for auto-read), and separate departmental tabs.
Can this run on engineer workstations without internet?	Soham (Architect)	Yes. The database is embedded SQLite, NLP models run local PyTorch inference, and OCR runs on local binaries. Zero cloud dependency.